Preface

Why This Guide Exists

Building intelligent AI systems in 2026 requires mastering an extraordinary breadth of knowledge — from how transformers process language internally, through the hardware and systems that make training possible, the optimization techniques that make it efficient, the reinforcement learning algorithms that teach models to reason and align with human intent, all the way to multi-agent architectures that coordinate autonomous systems at scale.

This knowledge is scattered across hundreds of papers, blog posts, GitHub repositories, and tribal knowledge within a handful of labs. This guide exists because **practitioners need a single, unified reference** that covers the entire stack — not just the theory, but the implementation details that make things actually work.

A Personal Journey to Agentic AI

My fascination with intelligent agents began two decades ago, when I still studied for my first degree in information systems engineering. I took courses on Agent-Oriented Software Engineering (AOSE) [382] and learned to build multi-agent systems using JADE [22] (Java Agent DEvelopment Framework)—a FIPA-compliant [93] platform where agents communicated via structured protocols, negotiated over shared resources, and coordinated autonomously. Around the same time, Berners-Lee, Hendler, and Lassila's seminal paper "The Semantic Web" [24] painted a vision of machine-readable knowledge that agents could reason over. These two threads—autonomous agent architectures and semantic knowledge representation—planted a seed that has guided my career ever since. One early project that crystallized this vision was an attempt to build a *shopping agent*—developed with OntoBuilder [103] under the guidance of my respected future academic advisor Prof. Avigdor Gal—a system that could automatically fill product search queries and orders across different heterogeneous websites, understanding their varied schemas through ontology matching and mapping. The Semantic Web promised that such agents would thrive in a world of structured, machine-readable data. In practice, the brittleness of hand-crafted ontologies, the messiness of real-world product data, and the lack of robust natural language understanding made the vision perpetually "five years away."

Over the following years, I tracked each wave of AI progress as it arrived: neural networks and heuristic search for combinatorial optimization; deep learning and representation learning; information retrieval and personalization at scale; and most recently, the revolution of large language models. Each wave brought powerful new tools, but the dream remained the same: systems that *understand*, *reason*, and *act* autonomously in complex environments.

What makes 2024–2026 remarkable is that these threads have finally converged. LLMs provide the language understanding and generation; reinforcement learning teaches them to reason and align with human intent; tool-use protocols (MCP) give them hands to act in the world; and agent orchestration frameworks provide the coordination layer that JADE envisioned twenty years ago—now powered by foundation models instead of hand-coded ontologies. This guide is, in many ways, the reference I wish I had at each step of that journey.

The Landscape in 2026

The journey to today's agentic AI systems spans three decades of compounding breakthroughs across architecture, training, and deployment:

1. **Architectural foundations (2017–2020)**: The Transformer [357] introduced self-attention as a universal sequence-processing primitive. Scaling laws revealed that larger models trained on more data reliably improve. GPT-2 and GPT-3 demonstrated that decoder-only transformers, scaled sufficiently, become capable few-shot learners.

2. **Systems and efficiency (2020–2023)**: Flash Attention [65] made training 2–4× faster by eliminating memory bottlenecks. LoRA [147] enabled fine-tuning 70B+ models on a single node. Mixture-of-Experts (MoE) decoupled model capacity from compute cost. Inference engines like vLLM brought throughput within reach of real-time applications.

3. **Alignment via RL (2022–2024)**: RLHF [280] transformed capable-but-unhelpful base models into useful assistants — the recipe behind ChatGPT. DPO [302] collapsed the reward model and RL loop into a single supervised loss, democratizing alignment. Variants proliferated: KTO [87], IPO [14], ORPO [141], GRPO [323].

4. **Reasoning and autonomy (2024–2026)**: DeepSeek-R1 [72] and OpenAI's o1/o3 demonstrated that RL could teach *reasoning itself* — models spontaneously discover chain-of-thought, backtracking, and self-verification. Simultaneously, the Model Context Protocol (MCP) standardized tool access, Agent-to-Agent (A2A) enabled inter-agent communication, and production-grade orchestration frameworks matured.

Who This Guide Is For

This document is written for **practitioners who build things**:

- **ML engineers** who need to understand transformer internals, training infrastructure, optimization, and why things diverge.
- **Applied researchers** evaluating architectures, fine-tuning strategies, and RL methods for their specific domains.
- **Agent developers** building production systems who need orchestration patterns, memory architectures, tool integration (MCP), and multi-agent coordination (A2A).
- **Systems engineers** responsible for training infrastructure, GPU clusters, distributed training, and inference deployment.
- **Technical leaders** making architectural and resourcing decisions across the full stack.

We assume familiarity with neural networks and basic probability. **No prior LLM, RL, or systems knowledge is required** — the guide builds from first principles.

What You Will Gain

After reading this guide, you will be able to:

- **Understand LLM internals** — attention mechanisms, positional encodings, MoE routing, Flash Attention, and why architectural choices matter for downstream capability.
- **Reason about systems** — GPU memory budgets, distributed training strategies (FSDP, tensor/pipeline parallelism), inference optimization, and production deployment with vLLM.

- **Train and fine-tune efficiently** — LoRA/QLoRA, quantization, knowledge distillation, optimizer selection, and learning rate scheduling.
- **Align models with human preferences** — implement RLHF/DPO/GRPO/KTO pipelines, debug reward hacking and mode collapse, choose the right algorithm among 20+ methods.
- **Build reasoning models** — understand how DeepSeek-R1, o1/o3, and QwQ discover chain-of-thought through RL without explicit demonstrations.
- **Architect agentic systems** — select orchestration patterns, design memory, integrate tools via MCP, coordinate agents via A2A, evaluate with production benchmarks.
- **Evaluate rigorously** — apply appropriate metrics, benchmarks, and LLM-as-Judge patterns for both model quality and agent capability.

How This Guide Is Organized

The guide spans 30 chapters organized in six parts:

1. **Part I — Foundations** (Chapters 1–3): LLM architecture and optimization (transformers, attention, positional encodings, Flash Attention, LoRA, MoE), systems foundations (GPU hierarchies, distributed training, vLLM), and classical RL theory (MDPs, policy gradients, actor-critic).
2. **Part II — RL Methods for LLMs** (Chapters 4–12): The complete RL-for-LLMs toolkit. RL foundations for language models, then full mathematical treatment of PPO, DPO, GRPO, and preference optimization variants (Online DPO, KTO, IPO, ORPO, SimPO), reward model training, SFT best practices, system architecture at scale, and agentic training with trajectory-level RL.
3. **Part III — Reasoning** (Chapter 13): Large reasoning models — DeepSeek-R1, OpenAI o1/o3/o4-mini, QwQ — how RL discovers chain-of-thought, MCTS, process reward models, and test-time compute scaling.
4. **Part IV — Evaluation** (Chapter 14): Comprehensive LLM evaluation methodology — metrics, LLM-as-Judge, human annotation, benchmark suites, contamination detection, and agentic evaluation.
5. **Part V — Agentic AI** (Chapters 15–27): The complete agentic stack — introduction to agentic AI, RAG and retrieval, memory systems, orchestration and context management, loop engineering, design patterns, agentic environments and benchmarks, Model Context Protocol (MCP), agent skills, Agent-to-Agent communication (A2A), multi-agent systems, development frameworks, and agentic UI.
6. **Part VI — Assessment & Reference** (Chapters 28–30): 108 detailed quiz questions with comprehensive answers spanning all topics, a quick-reference chapter consolidating key equations, API references, and failure mode diagnostics, and a conclusion with future directions.

The guide includes over 100 detailed quiz questions with comprehensive answers spanning all topics, plus a quick-reference chapter consolidating key equations, API references, and failure mode diagnostics.

Design Philosophy

Three principles guide this document:

1. **Intuition first, formalism second**. Every equation is preceded by a plain-English explanation of what it means and why it matters.
2. **Implementation-aware**. Theory is useless without knowing how to make it work. We include code, hyperparameter tables, memory budgets, architecture diagrams, and debugging strategies throughout.
3. **Honest about what works**. We clearly state which approaches are production-tested and which are research explorations.

Scope and Deliberate Omissions

This guide focuses on **text-in, text-out language models** and the RL, systems, and agentic infrastructure built around them. Several important areas are intentionally excluded:

- **Multimodal models** (vision–language, audio, video). Multimodal architectures introduce distinct training pipelines (contrastive pre-training, cross-modal alignment, modality-specific encoders), data curation challenges, and evaluation protocols that each merit book-length treatment. Including them would double the scope without deepening the RL and agentic core that unifies this guide.
- **Domain-specific deployments** (healthcare, legal, finance, scientific discovery). Domain adaptation introduces regulatory constraints, specialized evaluation, and data-access issues that are orthogonal to the general methods presented here. The algorithms and architectures we cover *are* the building blocks practitioners adapt to these domains, but the adaptation details are better served by dedicated references.
- **Personalization and recommendation systems.** Personalization relies on user modeling, collaborative filtering, and interaction-history architectures that form a parallel research tradition. While LLMs are increasingly used *within* recommender systems, the core techniques (sequential models, bandit-based exploration, cold-start handling) are sufficiently distinct to warrant separate coverage.

By maintaining this boundary, we keep a single coherent thread—*from architectural foundations and systems infrastructure, through the training algorithms that produce aligned and reasoning models, to the orchestration and deployment of autonomous agents*—without fragmenting the narrative across modalities and verticals.

— *Haggai Roitman, 2026*

What's New in Version 1.3

Version 1.3 adds nineteen new topics reflecting developments through mid-2026:

Chapter 1 — LLM Architecture and Optimization:

- **Muon optimizer** — Newton–Schulz orthogonalization as an AdamW successor (adopted by GLM-5, Kimi K2, DeepSeek-V4).
- **Multi-head Latent Attention (MLA)** — DeepSeek's KV-cache compression via low-rank latent projection.
- **FP8/FP4 training** — next-generation mixed precision with per-tile micro-scaling.
- **Auxiliary-loss-free MoE** — adaptive bias-based load balancing replacing costly auxiliary losses.
- **Mid-training** — the emerging stage between pre-training and SFT that prepares models for RL.

Chapter 2 — Systems Foundations:

- **Dynamo** — NVIDIA's datacenter-scale inference orchestration framework.

Chapter 11 — System Architecture at Scale:

- **Decoupled DiLoCo** — Google's geo-distributed training with 236× bandwidth reduction.
- **Miles** — PyTorch-native RL post-training engine.

Chapter 12 — LLM Agentic Training:

- **Interactive RL environments** — NeMo Gym, RLFactory, and MOSAIC for agentic post-training.
- **Training on real agent traces** — using production trajectories as RL training signal.

Chapter 13 — RL for Large Reasoning Models:

- **Inference-time compute scaling laws** — log-linear relationships and budget-aware prompting.
- **Latent reasoning** — coconut-style continuous thought without token-level chain-of-thought.
- **On-policy self-distillation** — dense token-level supervision from the model's own privileged context, achieving RL-level reasoning at 10–100× lower compute.

Chapter 17 — Agentic Memory Systems:

- **Proactive memory architectures** — Meta AI's behavioral-state decay and anticipatory retrieval.

Chapter 19 — Loop Engineering:

- **Context engineering** — the discipline of optimizing what fills the context window (Lütke, Karpathy, Anthropic).

Chapter 21 — Agentic Environments:

- **UniClawBench and Terminal-Bench** — 2026 benchmarks for robotic manipulation and long-horizon terminal tasks.

Chapter 25 — Multi-Agent Systems:

- **BDI-LLM self-evolving agents** — belief–desire–intention architectures with learned plan libraries.

Chapter 26 — Agent Development Frameworks:

- **NVIDIA OO Agents (NOOA)** — object-oriented framework where agents are Python objects, methods are tools, and `...` bodies become LLM-driven loops.